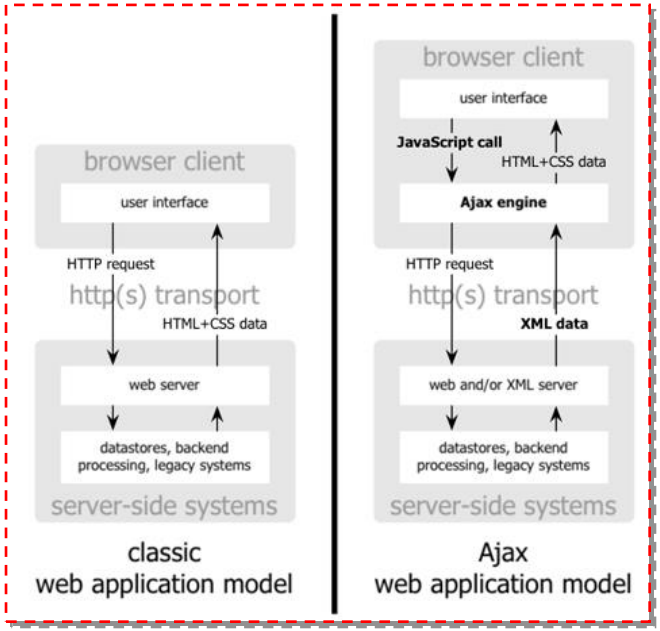


Introduction to Ajax



서강대학교
Sogang University

AJAX
(Asynchronous JavaScript and XML)

- **Why ?**
  - **Usability of web applications has lagged behind that of desktop applications**
  - **Rich Internet Applications (RIAs)**
    - ⇔ **Web applications that *approximate the look, feel and usability of desktop applications***
    - ⇔ **Two key attributes—*performance* and *rich GUI***
    - ⇔ **RIA performance**
      - ⇒ **Comes from Ajax (Asynchronous JavaScript and XML), which uses client-side scripting to make web applications *more responsive***
  - **Ajax applications separate *client-side user interaction* and *server communication*, and *run them in parallel*, making the delays of server-side processing more transparent to the user**
    - ⇔ **"Raw" Ajax uses JavaScript *to send asynchronous requests* to the server, *then updates the page using the DOM***
    - ⇔ **When writing "raw" Ajax you need to deal directly with cross-browser portability issues, making it impractical for developing large-scale applications**

- **Traditional web applications**
  - ① **User fills in the form's fields, then submits the form**
  - ② **Browser generates a request to the server, which receives the request and processes it**
  - ③ **Server generates and sends a response containing the exact page that the browser will render**
  - ④ **Browser loads the new page and temporarily makes the browser window blank**
  - ⑤ **Client *waits* for the server to respond and *reloads the entire page* with the data from the response**
  - *While a synchronous request is being processed on the server, the user cannot interact with the client web browser*
  - The **synchronous model** was originally designed for **a web of hypertext documents**

Synchronous

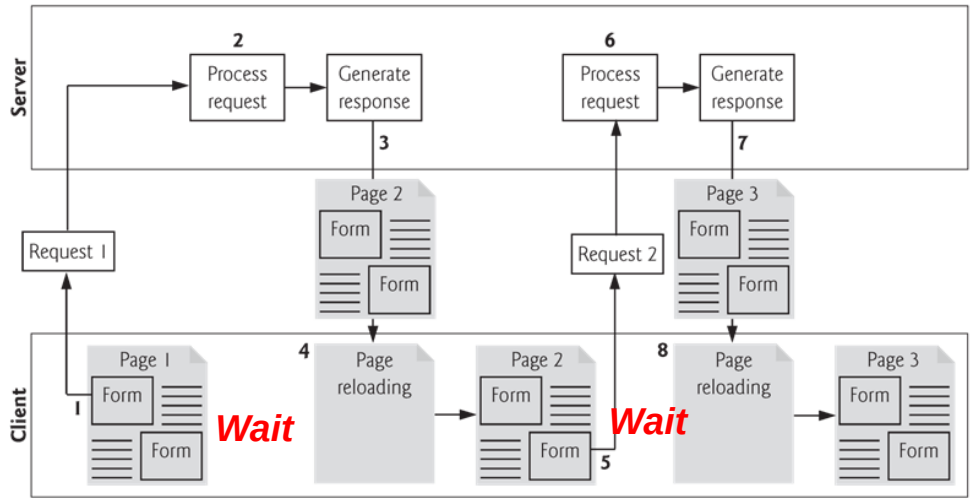


Fig. 16.1 | Classic web application reloading the page for every user interaction.

- **Ajax Web Applications**
  - In an Ajax application, when the user interacts with a page
    - ① Client creates an `XMLHttpRequest` object to manage a request
    - ② `XMLHttpRequest` object sends the request to and awaits the response from the server
    - ③ Requests are asynchronous, ***allowing the user to continue interacting with the application while the server processes the request concurrently***
    - ④ When the server responds, the `XMLHttpRequest` object that issued the request invokes a callback function, which typically uses ***partial page updates*** to display the returned data in the existing web page without reloading the entire page
  - Callback function updates ***only a designated part of the page***
    - ⇔***Partial page updates*** help make web applications more responsive, making them feel more like desktop applications

Asynchronous

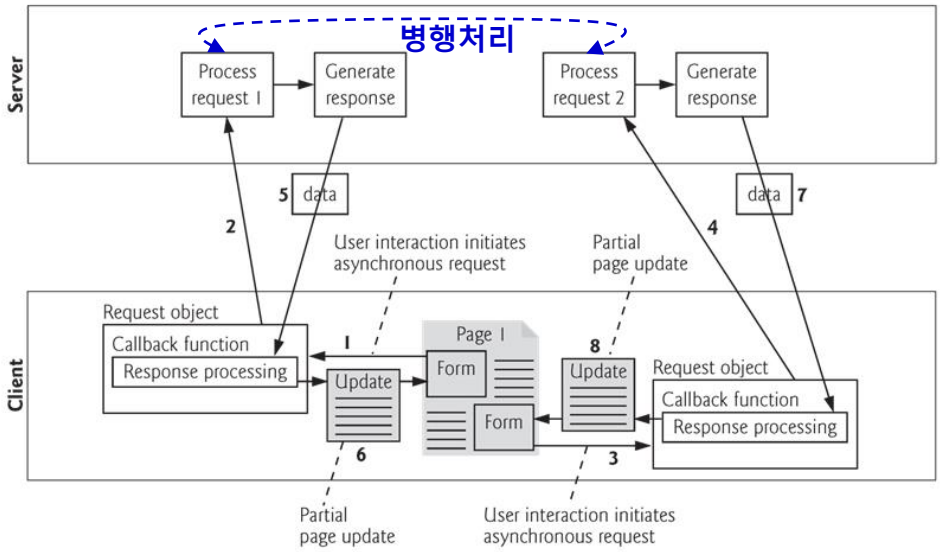
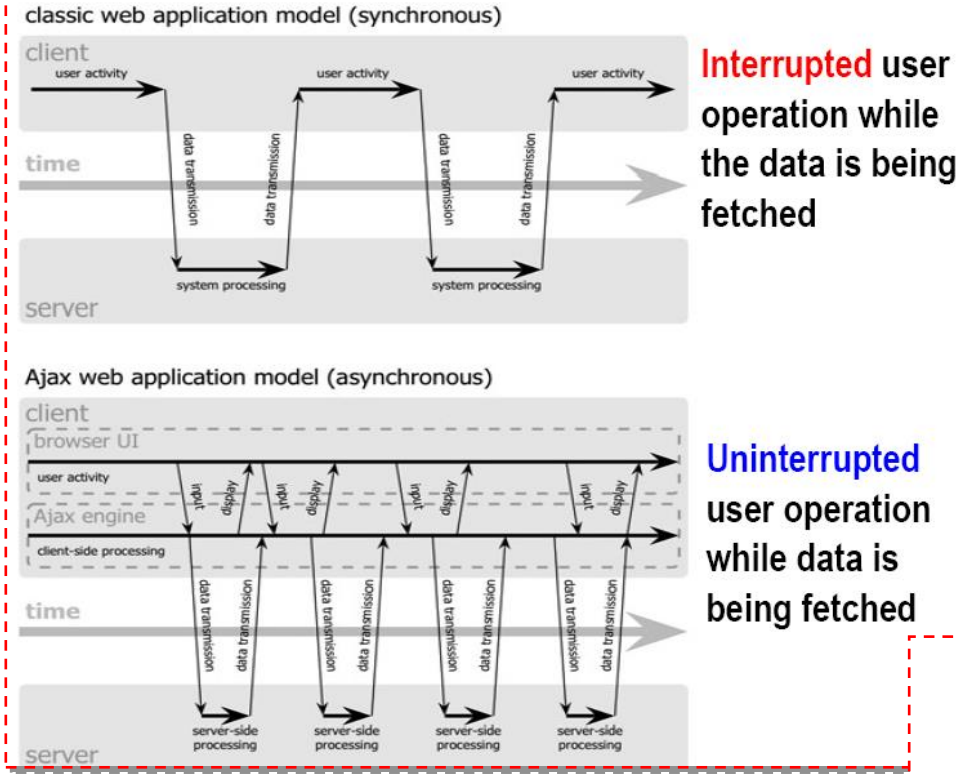
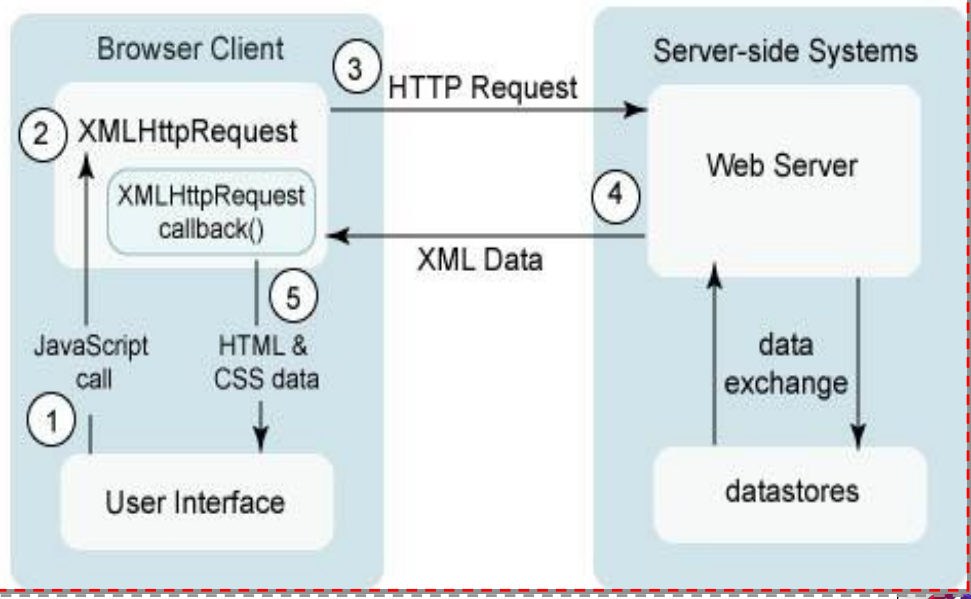


Fig. 16.2 | Ajax-enabled web application interacting with the server asynchronously.



Classic Web Application *vs.* Ajax Web Application



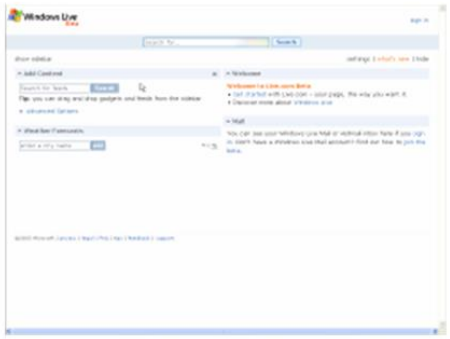
- **AJAX is Based on Internet Standards, and uses a combination of:**
  - XMLHttpRequest object (to exchange data asynchronously with a server)
  - JavaScript/DOM (to display/interact with the information)
  - CSS (to style the data)
  - XML (often used as the format for transferring data)

Examples From the Web

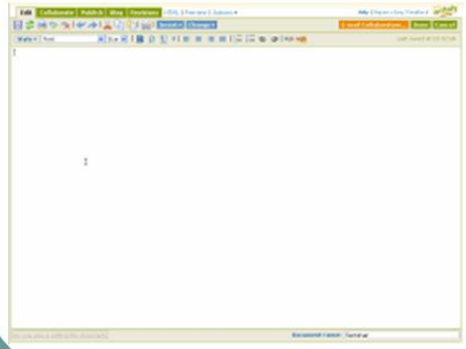
Google Suggest



Microsoft Live



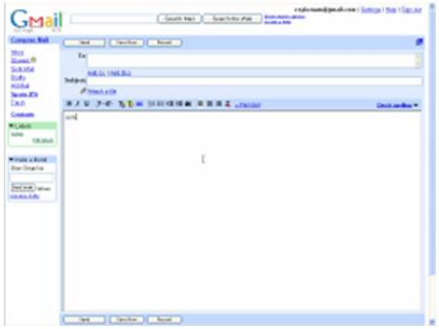
Writely



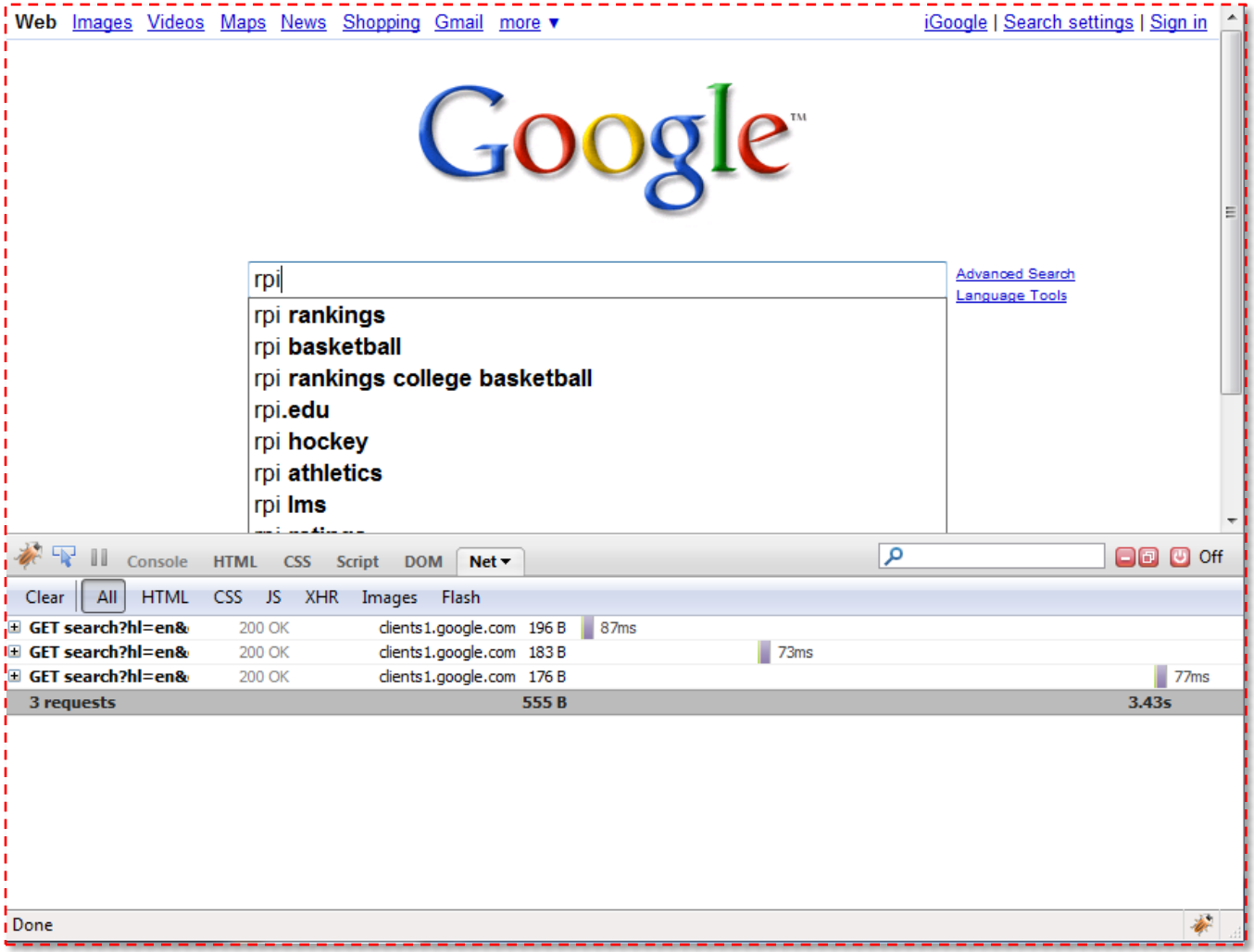
Google Maps



Gmail



Firefox


Web Images Videos Maps News Shopping Gmail more ▾
iGoogle | Search settings | Sign in
Google
rpi
rpi rankings
rpi basketball
rpi rankings college basketball
rpi.edu
rpi hockey
rpi athletics
rpi lms
Advanced Search
Language Tools
Console HTML CSS Script DOM Net
Off
Clear All HTML CSS JS XHR Images Flash
GET search?hl=en& 200 OK clients1.google.com 196 B 87ms
GET search?hl=en& 200 OK clients1.google.com 183 B 73ms
GET search?hl=en& 200 OK clients1.google.com 176 B 77ms
3 requests 555 B 3.43s
Done

Ajax: 새로고침 없이 웹 화면 일부를 갱신하기

Asynchronous JavaScript and XML — 비동기 HTTP 통신 + JavaScript + DOM 조작

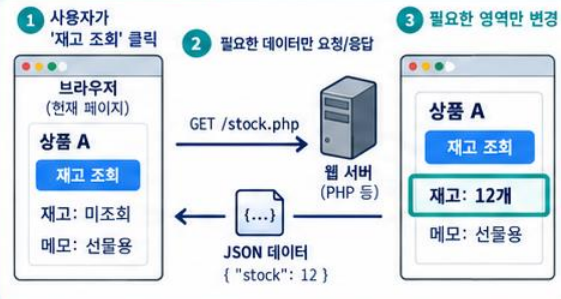
Ajax는 별도 언어나 프로토콜이 아닌 개발 방식
XML뿐 아니라 JSON·텍스트·HTML도 사용

1 등장 동기: 잦은 변화에도 전체 페이지를 다시 받아야 할까?

전체 페이지 이동·재로드

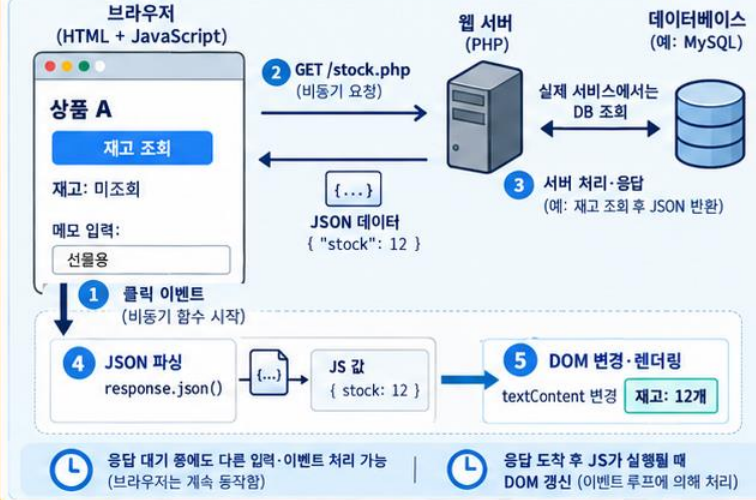


Ajax 방식



- 불필요한 데이터 전송 감소 가능
- 화면 상태 유지 (입력 내용, 스크롤 위치 등)
- 연속적인 사용자 경험 (더 부드러운 화면 전환)
- 일반 페이지 요청도 브라우저가 항상 멈추는 것은 아님: 핵심 차이는 **전체 문서 교체** vs **부분 갱신** 입니다.

2 실행 과정: 재고 조회 예제



- 응답 대기 중에도 다른 입력·이벤트 처리 가능 (브라우저는 계속 동작함)
- 응답 도착 후 JS가 실행될 때 DOM 갱신 (이벤트 루프에 의해 처리)

3 브라우저 코드: fetch() + async/await

```
1  <button id="check">재고 조회</button>
2  <p id="stock">재고: 미조회</p>
3  <script>
4    const btn = document.querySelector("#check");
5    const out = document.querySelector("#stock");
6    btn.addEventListener("click", async () => {
7      out.textContent = "조회 중...";
8      try {
9        const res = await fetch("/stock.php");
10       if (!res.ok) throw new Error("HTTP " + res.status);
11       const data = await res.json();
12       out.textContent = "재고: " + data.stock + "개";
13     } catch (err) {
14       out.textContent = "조회 실패";
15     }
16   });
17 </script>
```

fetch: 요청
지정한 URL로 비동기 HTTP 요청을 보냅니다.
응답은 Promise로 반환됩니다.

await: 이 함수의 후속 실행 대기
응답이 올 때까지 이 함수의 나머지 코드를 일시적으로 기다립니다.
await는 브라우저 전체를 멈추지 않음 (다른 이벤트·입력 처리는 계속 가능).

textContent: 일부 DOM 변경
서버에서 반은 값으로 특정 요소의 내용만 변경합니다 (전체 페이지를 다시 로드하지 않음).

4 서버 예제와 화면 결과

PHP: stock.php (서버 코드 예제)

```
1  <?php
2    header("Content-Type: application/json");
3    echo json_encode(["stock" => 12]);
```

설명용 고정 재고 값: 실제 서비스는 DB 등에서 조회하여 반환합니다.

브라우저 화면 결과



구현 API
- 전통적 XMLHttpRequest (과거 방식)
- 현대적 fetch (권장)

실무에서 알아둘 점
1. 같은 출처의 웹 서버에서 실행 (추가 설정 없이 호출 가능)
2. 다른 출처는 CORS 허용 필요 (브라우저의 보안 정책)
3. 서버의 응답만으로 DOM이 자동 변경되지는 않음 (반드시 JavaScript로 처리 필요)

사용자 이벤트 → 비동기 HTTP → 데이터 응답 → DOM 일부 수정 → 화면 갱신

참고: MDN XMLHttpRequest / Network requests

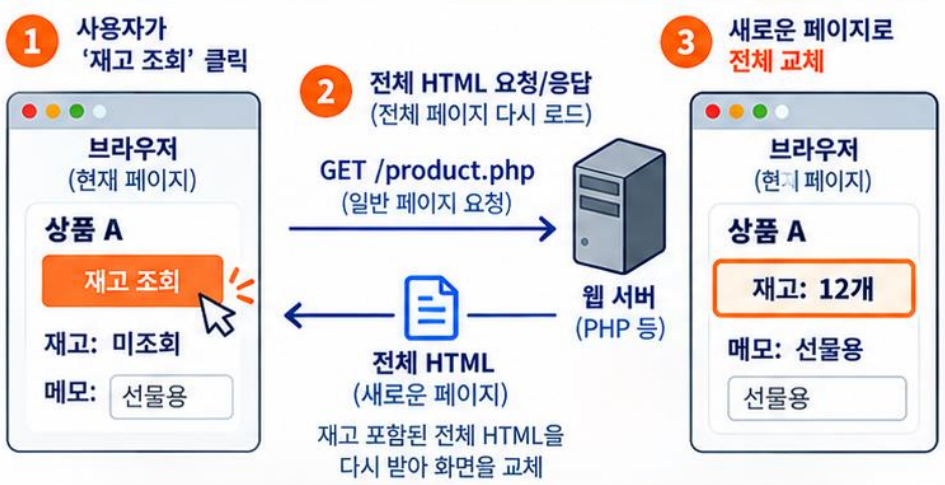
1/4

Ajax ① 등장 동기: 전체 재로드에서 부분 갱신으로

작은 변화에도 전체 페이지를 다시 받는 대신, 필요한 데이터만 받아 화면의 일부만 갱신하는 방식

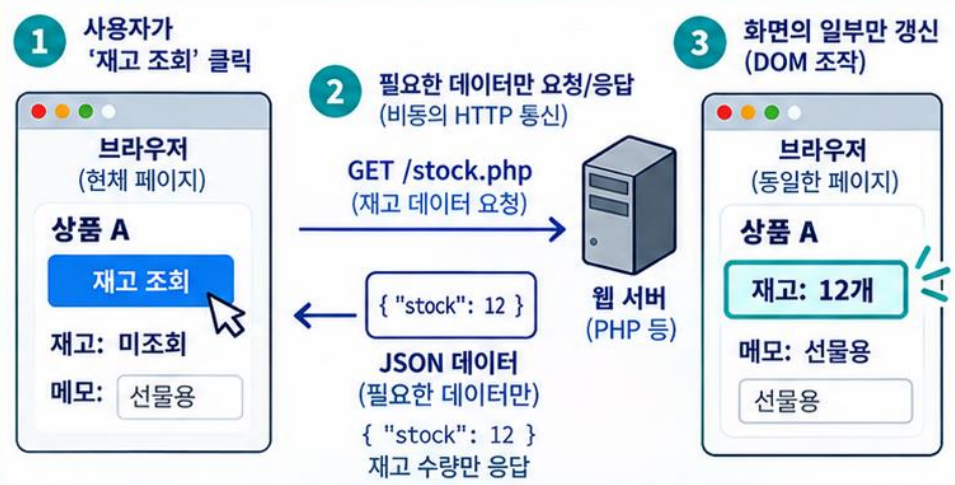
기존 방식: 전체 HTML 페이지 이동 · 재로드

재고만 보려 해도 전체 페이지를 다시 받아야 함



Ajax 방식: 필요한 데이터만 요청 · 부분 갱신

재고 데이터만 받아 화면의 일부만 갱신



불필요한 데이터 전송을 줄일 수 있음

재고처럼 일부 데이터만 요청하여
네트워크 트래픽과 응답 시간을 줄일 수 있습니다.

현재 화면의 상태를 유지할 수 있음

사용자가 입력한 내용, 스크롤 위치 등
현재 페이지의 상태가 그대로 유지됩니다.

더 빠르고 부드러운 사용자 경험

전체 페이지를 다시 로드하지 않아
끊김 없이 빠르게 응답하는 인터랙션이 가능합니다.

일반적인 페이지 이동(링크 클릭, 폼 제출 등)이 반드시 브라우저를 멈추게 하는 것은 아닙니다.

브라우저는 다른 작업을 계속 처리할 수 있습니다. Ajax의 핵심 차이는 "전체 문서 교체"가 아니라 "필요한 데이터를 받아 DOM의 일부만 갱신하는 것"입니다.

Ajax = Asynchronous JavaScript and XML

- ✓ 비동기 HTTP 통신을 이용해 서버와 데이터를 주고받고,
- ✓ 받은 데이터를 JavaScript로 화면에 반영하는 기법입니다.
- ✓ Ajax는 별도의 언어나 프로토콜이 아닌 **개발 방식(기술)**이며, 반드시 XML만 사용하는 것이 아니라 **JSON, 텍스트**, HTML 등 다양한 형식의 데이터를 사용할 수 있습니다.

필요한 데이터만 받아 JavaScript로 화면 일부를 갱신

2/4 Ajax ② 동작 과정: 클릭에서 화면 갱신까지

비동기 HTTP 통신으로
일부만 갱신 (전체 페이지 이동 없음)

브라우저 (HTML + JavaScript)

http://example.com

상품 A

재고 조회 1

재고: 미조회

메모: 선물용

5 out.textContent = "재고: 12개"
DOM 변경 (JS가 직접 처리)

상품 A

재고 조회

재고: 12개

메모: 선물용

웹 서버 (PHP)

2 fetch("/stock.php")
HTTP GET 요청
(비동기, 전체 페이지 이동 없음)

```
GET /stock.php HTTP/1.1
Host: example.com
Accept: application/json
```

stock.php

```
<?php
// DB에서 재고 조회
$stock = 12;
echo json_encode([
  "stock" => $stock
]);
?>
```

4 await response.json()
응답 파싱 (JS 객체로 변환)

```
{ "stock": 12 }
```
(JavaScript 객체)

데이터베이스 (예: MySQL)

3 서버 처리 · DB 조회
(예: 재고 조회 후 JSON 변환)

```
SELECT stock
FROM products
WHERE id = 1;
```

```
{ "stock": 12 }
```

비동기 처리 타임라인 (await 동안 무슨 일이 일어날까?)

시간 흐름 →

JS 이벤트 핸들러 (클릭 함수)
1 클릭 이벤트 핸들러 시작
2 fetch() 호출 후 await로 대기
await 동안 이 함수는 일시 중단 (다른 코드 실행 가능)
4 응답 도착 후 다시 실행 (responce.json())
5 DOM 변경 (textContent)

브라우저 (다른 작업 + 렌더링)
네트워크 요청이 진행되는 동안
사용자 다른 입력 처리, UI 렌더링 동이 계속 실행됨
(브라우저는 멈추지 않음)

응답이 와도 화면은 자동으로 바뀌지 않습니다.
fetch()는 데이터를 가져올 뿐이며,
화면 변경은 JS가 DOM을 직접 수정해야 합니다.

HTTP 통신(네트워크)과 DOM(메모리)은 다릅니다.
- HTTP: 서버와 데이터를 주고받는 네트워크 통신
- DOM: 브라우저 메모리에 있는 현재 화면 구조
(JS가 수정해야 화면에 반영됨)

- 전체 페이지 이동(새로고침) 없음
- 필요할 때 1회 요청 (폴링 아님)
- 다른 이벤트 처리와 화면 렌더링은 계속 가능

전체 흐름: 이벤트 발생 → 비동기 HTTP 통신 → 데이터 처리 → DOM 변경 → 렌더링 (일부 화면만 갱신)

3 Ajax ③ 브라우저 프로그래밍: fetch() + async/await

브라우저 HTML + JavaScript 코드

```
1  <button id="check">재고 조회</button>
2  <p id="stock">재고: 미조회</p>
3  <script>
4     const btn = document.querySelector("#check");
5     const out = document.querySelector("#stock");
6     btn.addEventListener("click", async () => {
7        out.textContent = "조회 중...";
8        try {
9           const res = await fetch("/stock.php");
10          if (!res.ok) throw new Error("HTTP " + res.status);
11          const data = await res.json();
12          out.textContent = "재고: " + data.stock + "개";
13       } catch (err) {
14          out.textContent = "조회 실패";
15       }
16    });
17 </script>
```

코드 동작 설명 (숫자와 코드 줄이 대응)

1 addEventListener로 클릭 핸들러 등록 (6행)
버튼이 클릭되면 async 함수가 실행됩니다.

2 fetch()로 GET 요청 시작 (9행)
fetch("/stock.php")가 비동기 HTTP GET 요청을 보내고 Promise를 반환합니다.

3 첫 번째 await로 응답 객체 획득 (9행)
await이 Promise가 끝날 때까지 이 함수의 실행만 일시적으로 기다리고, 브라우저 전체가 멈추지 않습니다.

4 res.ok로 HTTP 상태 확인 (10행)
응답이 성공(200번대)이 아니면 에러를 발생시켜 catch 블록으로 이동합니다.

5 두 번째 await로 JSON 본문 파싱 (11행)
await res.json()으로 응답 본문을 읽고 JSON으로 변환합니다.

6 textContent로 화면 업데이트 (12행, 14행)
성공 시 "재고: 12개"를 표시하고, 실패 시 "조회 실패"를 표시합니다.

화면 실행 결과 (브라우저에서의 변화)



요청과 응답 과정



중요한 점

- **await은 함수가 아니라 연산자입니다.**
  Promise가 끝날 때까지 현재 async 함수의 실행만 기다립니다. 브라우저 전체가 멈추지 않습니다.
- **같은 출처의 /stock.php를 요청합니다.**
  브라우저와 동일한 도메인에 있는 서버 엔드포인트여야 하며, { "stock": 12 } 형식의 JSON을 반환하는 서버가 반드시 필요합니다.

4 Ajax ④ 서버 응답과 화면 결과

웹 서버 (PHP): stock.php (서버 코드 예제)




```php
<?php
header("Content-Type: application/json");
echo json_encode(["stock" => 12 ]);
?>
```

설명용 고정 재고 값입니다.
실제 서비스는 DB에서 조회하고 입력값을 검증합니다.

서버 → 브라우저: HTTP 응답 (일부 해더만 표시)

JSON 응답 전송 (HTTP)

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "stock": 12 }
```

데이터 전달 (JSON)

브라우저 (웹 페이지)

상품 A
재고 조회
재고: 12개
메모: 선물용

브라우저의 JS 처리 (async 이벤트 핸들러의 일부)

```javascript
const res = await fetch("/stock.php");
const data = await res.json();
out.textContent = "재고: " + data.stock + "개";
```

out은 화면의 재고를 표시하는 요소(예: id="stock")를 가리킵니다.
async/await를 사용한 이벤트 처리 함수의 일부 코드입니다.

브라우저 화면 결과

요청 전 (BEFORE)

상품 A
재고 조회
재고: 미조회
메모: 선물용

같은 페이지 새로고침 없음

응답 후 (AFTER)

상품 A
재고 조회
재고: 12개
메모: 선물용

구현 API와 주의사항

1. **전통적 XMLHttpRequest**
   - 과거 방식의 비동기 요청
2. **현대적 fetch (권장)**
   - 더 간단하고 현대적인 방식
3. **동일 출처 가정 (Same-Origin)**
   - 예제는 같은 출처의 웹 서버에서 실행되는 경우를 가정
   - 다른 출처의 서버를 호출하려면 CORS 설정이 필요

응답 데이터는 화면을 자동으로 바꾸지 않음

- 서버는 데이터(JSON)만 응답할 뿐, 웹 페이지의 화면은 자동으로 갱신되지 않습니다.
- 응답을 받은 후 JavaScript가 DOM을 변경해야 화면에 반영되고, 브라우저가 이를 렌더링합니다.

PHP의 json_encode() → JSON 응답 → res.json()으로 데이터 처리 → DOM 일부 수정 → 화면 갱신

<실습> cspro.sogang.ac.kr의 /tmp/jhnang/stock_check.html 과 /tmp/jhnang/cgi-bin/stock.php 를 자신의 홈 디렉토리로 복사하여 수행시켜 볼 것. (메모도 나오게 수정, stock_check2.html, stock2.php)

fetch()와 await: 비동기 요청을 순서대로 다루기

fetch가 작업을 시작하고, await가 결과를 받은 뒤 후속 코드를 재개한다.

fetch()는 함수 / await는 함수가 아닌 연산자: await fetch(url)
✕ await()라는 함수가 아님!

① fetch(): HTTP 요청 시작



fetch()가 반환하는 값

Promise<Response>

pending (대기 중) → fulfilled (Response)

네트워크 오류 등 → rejected (거부)

```
const promise = fetch("/stock.php");
```

- 호출 후 Promise를 반환하므로 다음 코드가 계속 실행될 수 있음.
- 응답 상태·헤더가 준비되면 Response를 얻음: 본문 전체 수신 완료와는 다름.
- GET이 기본값이며, options로 POST · headers · body 등을 지정할 수 있음.

② await: 현재 async 함수의 후속 실행 대기

```
const res = await fetch("/stock.php");
```

Promise를 기다림 (await) → 이행: 결과값을 res에 대입 (Response 객체) → 거부: 예외로 전달 (예: 네트워크 오류)

load 함수 (예: async function load() {…})	요청 (fetch 호출)	이 함수의 후속 실행 대기 (suspended)	결과 처리 (이후 코드 재개의)
브라우저 메인 스레드 (이벤트 루프)	다른 이벤트 처리 · 렌더링 가능 (블로정되지 않음) →		

- await는 async 함수 안 또는 모듈 최상위에서 사용할 수 있음.
- async 함수의 호출은 항상 Promise를 반환함.
- **await가 작업을 시작하는 것은 아님: 여기서는 fetch 호출이 요청을 시장함.**

③ 왜 await가 두 번 필요한가?

```
async function load() {
  const res = await fetch("/stock.php");
  if (!res.ok) throw new Error("HTTP " + res.status);
  const data = await res.json();
  document.querySelector("#stock").textContent =
    "재고: " + data.stock + "개";
}
load().catch(console.error);
```

첫 await: Response 객체 얻기 (응답 상태 · 헤더 확인 가능)

둘째 await: 응답 본문 읽기 · JSON 파싱 결과 얻기 (JS 객체 data를 얻음)

실행 컨텍스트
재고: 미조회 생성 후 실행

서버 응답 예시 (/stock.php)
{ "stock": 12 }

브라우저 화면 (실행 전 → 후)
재고: 미조회 → 재고: 12개

- res.json()도 Promise를 반환하며, 결과는 JSON 문저열이 아닌 JS 값(객체 등)임.

④ 실행 순서 확인

```
async function demo() {
  console.log("A");
  await Promise.resolve();
  console.log("C");
}

demo();
console.log("B");
```

콘솔 출력 결과

A → B → C

- await 뒤의 코드는 나중에 Microtask로 재개됨.
- 이미 이행된 Promise라도 후속 코드는 비동기적으로 재개됨.

HTTP 오류 확인
404 · 500도 fetch Promise는 이행될 수 있음
→ res.ok 검사.

await 없이 결과 사용?
const p = fetch(url); // p는 Response가 아니라 Promise
await 또는 .then()으로 결과 처리.

fetch = 비동기 요청 / **await** = 결과를 기다린 뒤 현재 함수의 실행 재개

Promise는 "나중에 끝날 작업의 결과 또는 실패를 나타내는 객체"입니다. 결과를 나중에 받기 위한 **접수증**이라고 생각하면 쉽습니다. MDN

JavaScript

```
const p = fetch("/stock.php"); // 요청 시작 → Promise 반환
const res = await p;           // 기다려서 Response 객체 얻기
const data = await res.json(); // 본문을 읽고 JSON 파싱
```

Ajax 입력값 전송: fetch()의 GET vs POST

입력 요소의 value 읽기 → 요청 구성 → 서버 수신 → 응답으로 DOM 갱신

1 예제 화면과 입력값 읽기 (공통)



브라우저 (현재 페이지)

HTML (일부)

```html
<input id="name" value="Kim">
<input id="age" type="number" value="20">
<button id="getBtn">GET 조회</button>
<button id="postBtn">POST 등록</button>
<p id="result"></p>
```

입력값 읽기 (JavaScript)

```js
const name = document.querySelector("#name").value;
const age = Number(document.querySelector("#age").value);
```

아래 코드는 각 버튼의 async 이벤트 핸들러 안에서 실행하는 핵심 부분입니다.

- 버튼을 클릭할 때 입력 요소의 value를 읽습니다. (페이지 로드 시 한 번만 읽지 않음)
- 버튼의 id가 위와 같다면 별도의 이벤트 리스너 코드는 생략합니다.

예제 입력값

```
{
  name: 'Kim',
  age: 20
}
```

2 GET: URL 쿼리로 전달 / 조회·검색 (데이터 조회에 사용)

JavaScript (fetch)

```js
const qs = new URLSearchParams({ name, age });
const res = await fetch("/user.php?" + qs);
// method 기본값은 GET
```

- URLSearchParams가 특수문자·한글 인코딩 처리
- fetch의 GET에는 body를 넣지 않음

요청 데이터 (HTTP 요청)

```
GET /user.php?name=Kim&age=20 HTTP/1.1
```



PHP (user.php: 쿼리 값 수신)

```php
$name = $_GET["name"] ?? "";
$age = (int)($_GET["age"] ?? 0);
```

$_GET 배열로 URL의 쿼리 값을 전달받습니다.

3 POST: 요청 본문으로 전달 / 등록·처리 요청 (데이터 제출에 사용)

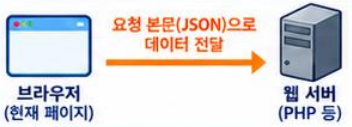
JavaScript (fetch)

```js
const res = await fetch("/user.php", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name, age })
});
```

- 요청 본문에 JSON 형식으로 데이터 전달
- JSON은 POST의 필수 형식의 아니며, 상황에 따라 FormData, URLSearchParams도 사용할 수 있습니다.

요청 데이터 (HTTP 요청)

```
POST /user.php HTTP/1.1
Content-Type: application/json
{"name":"Kim","age":20}
```



PHP (user.php: JSON 본문 수신)

```php
$data = json_decode(
  file_get_contents("php://input"), true
);
$name = $data["name"] ?? "";
$age = (int)($data["age"] ?? 0);
```

JSON 본문은 $_POST가 아니라 php://input으로 읽습니다.

4 공통 응답 처리: 받은 값으로 화면 일부 갱신

PHP (두 방식의 수신 코드 뒤에 실행)

```php
header("Content-Type: application/json");
echo json_encode(["message" => $name . " 수신 완료"]);
```

응답 데이터 (HTTP 응답)

```
{
  "message": "Kim 수신 완료"
}
```



브라우저 (응답으로 DOM 갱신)

JavaScript (각 fetch 코드 다음에 실행)

```js
if (!res.ok) throw new Error("HTTP " + res.status);
const data = await res.json();
document.querySelector("#result").textContent = data.message;
```

예제: Kim 수신 완료 / 오류는 이벤트 핸들러의 try/catch 등으로 처리합니다.

5 GET vs POST 비교 요약

항목	GET (URL 쿼리로 전달)	POST (요청 본문으로 전달)
데이터 위치	URL 쿼리 (?name=Kim&age=20)	요청 본문(JSON, 폼 데이터 등)
주요 의도	데이터 조회 (상태 변경 금지)	데이터 제출 · 상태 변경 등
노출 · 크기	URL, 서버 로그 등에 남을 수 있음 / URL 길이 제약	URL에 본문은 안 보이다 암호화는 아님 / 서버 크기 제한
PHP 수신	$_GET	JSON: php://input / 폼 인코딩: $_POST

- GET · POST 모두 비동기(비등기 요청)로 사용할 수 있습니다.
- 둘 다 HTTPS를 사용하는 것이 좋습니다.
- fetch는 주소창의 페이지를 이동시키지 않고, 현재 페이지에서 비동기로 데이터만 주고받습니다.
- POST는 FormData · URLSearchParams로도 전송할 수 있습니다.
- JSON은 선택한 본문 형식일 뿐 필수는 아닙니다.
- 서버에서는 받은 입력값을 반드시 검증해야 합니다.

예제는 같은 출처 (Same-Origin)의 서버를 가정합니다. (예: 동일한 도메인)

```html
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>Ajax GET·POST 예제</title>
</head>
<body>
  이름 : <input id="name" value="Kim">
  나이 : <input id="age" type="number" value="20">

  <button id="getBtn">GET 조회</button>
  <button id="postBtn">POST 등록</button>

  <p id="result"></p>

  <script>
    // 공통 : 응답을 읽고 화면 변경
    async function showResult(res) {
      if (!res.ok) {
        throw new Error("HTTP " + res.status);
      }

      const data = await res.json();
      document.querySelector("#result").textContent = data.message;
    }

    // GET: URL 쿼리로 입력값 전송
    document.querySelector("#getBtn")
      .addEventListener("click", async () => {
        const name = document.querySelector("#name").value;
        const age = Number(document.querySelector("#age").value);

        try {
          const qs = new URLSearchParams({ name, age });
          const res = await fetch("/user.php?" + qs.toString());

          await showResult(res);
        } catch (err) {
          document.querySelector("#result").textContent = "요청 실패";
          console.error(err);
        }
      });
```

cspro.sogang.ac.kr/tmp/jhnang/ajax_get_post.html

```
    // POST: JSON 본문으로 입력값 전송
    document.querySelector("#postBtn")
      .addEventListener("click", async () => {
        const name = document.querySelector("#name").value;
        const age = Number(document.querySelector("#age").value);

        try {
          const res = await fetch("/user.php", {
            method: "POST",
            headers: {
              "Content-Type": "application/json"
            },
            body: JSON.stringify({ name, age })
          });

          await showResult(res);
        } catch (err) {
          document.querySelector("#result").textContent = "요청 실패";
          console.error(err);
        }
      });
  </script>
</body>
</html>
```

```php
<?php
header("Content-Type: application/json; charset=UTF-8");

if ($_SERVER["REQUEST_METHOD"] === "GET") {
    // GET: URL 쿼리에서 읽기
    $name = $_GET["name"] ?? "";
    $age = (int)($_GET["age"] ?? 0);

} elseif ($_SERVER["REQUEST_METHOD"] === "POST") {
    // POST: JSON 요청 본문에서 읽기
    $data = json_decode(
        file_get_contents("php://input"),
        true
    );

    $name = $data["name"] ?? "";
    $age = (int)($data["age"] ?? 0);

} else {
    http_response_code(405);
    header("Allow: GET, POST");

    echo json_encode(["message" => "지원하지 않는 요청 방식"]);
    exit;
}

// 공통 응답
echo json_encode(
    ["message" => $name . "수신 완료"],
    JSON_UNESCAPED_UNICODE
);
```

cspro.sogang.ac.kr/tmp/jhnang/cgi-bin/user.php

<참고 1>

URLSearchParams: URL 쿼리를 쉽게 만들고 읽기

이름-값 쌍을 관리하고, URL·폼 형식에 맞게 인코딩하는 Web API

보통 new URLSearchParams(..)로 객체 생성
HTTP 요청 함수는 아님

어떻게 변환될까?

`{ q: 'Web & AI', page: 2 }` → `new URLSearchParams(…)` → `params.toString()` → `q=Web+%26+AI&page=2` → `/search.php?q=Web+%26+AI&page=2`

- 자바스크립트 객체 (이름-값 쌍)
- URLSearchParams 객체 생성 (이름-값 쌍을 관리)
- 쿼리 문자열로 변환 (URL·폼 형식에 맞게 인코딩)
- toString() 결과에는 앞의 ?가 없음

완성된 URL 예시: `/search.php?q=Web+%26+AI&page=2`

- ? 쿼리 시작 (query)
- & 필드 구분 (field separator)
- = 이름과 값 구분 (name / value)

- 'Web & AI'의 '&'는 %26으로 인코딩
- 공백은 +로 변환
- page: 2는 문자열 '2'로 변환

1 생성·읽기 쿼리 문자열을 파싱하여 값 읽기

```
const p = new URLSearchParams("q=Web&page=2");

p.get("q");       // "Web"
p.get("page");    // "2" (문자열)
p.get("missing"); // null
```

현재 페이지의 쿼리 사용 예시

```
const current = new URLSearchParams
  (location.search);
current.get("q");
```

전체 URL이라면 new URL(url).searchParams 사용

입력으로는 쿼리 문자열만 넣어야 합니다. 전체 URL이 아니라 'q=Web&page=2' 같은 쿼리 부분을 전달하세요.

2 수정·추가·반복 값 값을 추가·수정하고, 같은 이름의 값을 여러 개 가질 수 있음

```
const p = new URLSearchParams();
p.append("tag", "AI");
p.append("tag", "Web");
p.get("tag");       // "AI"
p.getAll("tag");    // ["AI", "Web"]
p.set("tag", "JS"); // 해당 이름의 기존 값을 하나로 교체
p.toString();       // "tag=JS"
p.has("tag");       // true
p.delete("tag");    // 해당 이름의 값 삭제
```

- **append:** 같은 이름의 값을 추가 (반복 값을 가질 수 있음)
- **set:** 해당 이름의 기존 값을 하나로 교체

3 fetch GET: URL에 붙이기 생성한 쿼리 문자열을 URL에 붙여 전송

HTML 입력 (이미 존재한다고 가정)

```
<input id="q" value="Web & AI">
```

async 이벤트 핸들리 내부

```
const q = document.querySelector("#q").value;
const p = new URLSearchParams({ q, page: 2 });
const res = await fetch("/search.php?"+ p.toString());;
const data = await res.json();
```

브라우저 (클라이언트) → GET /search.php?q=Web+%26+AI&page=2 → PHP 서버

PHP에서 값 읽기

```
$q = $_GET['q'] ?? '';
// 디코딩되어 'Web & AI' (원래 값)
```

이미 쿼리가 있는 URL은 URL 객체의 searchParams.set()를 사용하세요.
예) `const u = new URL("/search.php?sort=asc", location.href); u.searchParams.set("q", q); fetch(u);`

4 fetch POST: 본문에 넣기도 가능 폼 형식으로 본문을 전송

```
const p = new URLSearchParams({
  name: "Kim",
  age: 20
});
const res = await fetch("/user.php", {
  method: "POST",
  body: p
```

요청 내용 (브라우저가 자동으로 설정)

```
Content-Type: application/x-www-form-urlencoded;charset=UTF-8

name=Kim&age=20
```

PHP 서버

PHP에서 값 읽기

```
$name = $_POST['name'] ?? '';
// 'Kim'
```

URLSearchParams 객체를 body로 주면 Content-Type이 자동으로 설정됩니다.
PHP: $_POST['name']로 수신 가능.

JSON 본문과 다름: JSON.stringify() 불필요.

res.ok 및 오류 처리는 생략한 핵심 코드입니다.

꼭 알아둘 점

1. **원본 값을 전달** 미리 encodeURIComponent()하면 이중 인코딩될 수 있음.
2. **문자열 입력의 +는 공백으로 해석** 쿼리 문자열에서 +는 공백으로 인식됩니다. 실제 + 문자는 append()/set()에 원본으로 전달하세요.
3. **암호화가 아닌 인코딩** URLSearchParams는 암호화가 아니라 인코딩입니다. 숫자·불리언도 문자열로 관리됩니다.
4. **한글도 퍼센트 인코딩 처리** 한글도 자동으로 퍼센트 인코딩됩니다. 증첩 객체·파일은 다른 표현 방식(에: FormData) 검토가 필요합니다.

입력값 → URLSearchParams → 쿼리 문자열 또는 폼 본문 → fetch 전송

출처 MDN Web Docs – URLSearchParams https://developer.mozilla.org/docs/Web/API/URLSearchParams

<참고 2>

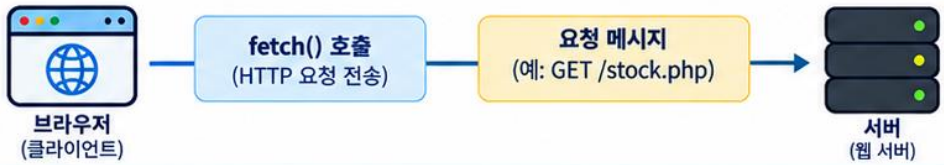
fetch()와 GET: 도구와 요청 메서드의 차이

fetch()는 JavaScript의 HTTP 요청 함수이고, GET은 HTTP 요청 메서드이다.

**fetch()는 함수(요청을 보내는 도구)이고,
GET은 그 요청이 무엇을 하려는지 나타내는 메서드이다.**

1 fetch(): JavaScript의 HTTP 요청 함수

fetch()는 브라우저에서 제공하는 JavaScript 함수로, HTTP 요청을 보내고 응답을 `Promise<Response>` 로 반환한다.



주요 특징

- Promise<Response>를 반환하며, 비동기적으로 동작한다.
- GET, POST, PUT, DELETE 등 다양한 HTTP 메서드로 요청을 보낼 수 있다.
- URL, 메서드, headers, body 등 다양한 옵션을 지정할 수 있다.

2 GET: HTTP 요청 메서드

GET은 HTTP 요청 메서드 중 하나로, 특정 리소스의 표현(데이터)을 요청하여 가져오는 데 사용된다. 주로 조회·검색에 사용되며, **서버의 상태를 변경하지 않아야 한다.**



3 fetch() 사용 예: 다양한 HTTP 메서드

1) 기본 GET 요청
(옵션 생략 시 자동으로 GET)

```
fetch("/stock.php")
  .then(res => res.json())
  .then(data => {
    console.log(data);
  })
  .catch(err => {
    console.error(err);
  });
```

2) 명시적 GET 요청
(method: "GET")

```
fetch("/stock.php", {
  method: "GET"
})
  .then(res => res.json())
  .then(data => {
    console.log(data);
  });
```

3) POST 요청 (JSON 데이터 전송)

```
fetch("/users.php", {
  method: "POST",
  headers: {
    "Content-Type":
      "application/json"
  },
  body: JSON.stringify({
    name: "Kim"
  })
});
```

4 fetch()와 GET의 관계

분류	JavaScript 함수 (fetch)	HTTP 메서드 (GET 등)
역할	요청 전송·응답 Promise 제공	요청의 의미 지정 (리소스를 가져올지, 추가할지 등)
관계	GET · POST · PUT · DELETE 등 메서드를 선택하여 보낼 수 있음	fetch뿐만 아니라 링크, 폼, 다른 클라이언트로도 보낼 수 있음

5 다양한 방법으로 보내는 GET과 POST 요청

1 링크 클릭 → GET 요청

```
<a href="/stock.php">주가 보기</a>
```



2 fetch() 기본 호출 → GET 요청

```
fetch("/stock.php");
```



3 fetch({ method: "POST" }) → POST 요청

```
fetch("/users.php", { method: "POST", ... });
```



6 핵심 정리

**fetch와 GET은 대체 관계가 아니다:
fetch로 GET 요청을 보낸다.**

- fetch()는 HTTP 요청을 보내는 JavaScript 함수(도구)이다.
- GET은 요청의 의미를 지정하는 HTTP 메서드이다.
- fetch()의 기본 메서드는 GET이다.
- fetch()의 Promise / 비동기 동작은 메서드(GET/POST)와 별개의 개념이다.
- GET은 데이터를 조회·검색하는 용도로 사용하며, 서버의 상태를 변경하지 않아야 한다.

출처: MDN Fetch API | MDN HTTP methods

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>Ajax GET·POST 예제</title>
</head>
<body>
  이름: <input id="name" value="Kim">
  나이: <input id="age" type="number" value="20">

  <button id="getBtn">GET 조회</button>
  <button id="postBtn">POST 등록</button>

  <p id="result"></p>

  <script>
    // 공통: 응답을 읽고 화면 변경
    async function showResult(res) {
      if (!res.ok) {
        throw new Error("HTTP " + res.status);
      }

      const data = await res.json();
      document.querySelector("#result").textContent = data.message;
    }

    // GET: URL 쿼리로 입력값 전송
    document.querySelector("#getBtn")
      .addEventListener("click", async () => {
        const name = document.querySelector("#name").value;
        const age = Number(document.querySelector("#age").value);

        try {
          const qs = new URLSearchParams({ name, age });
          const res = await fetch("/user.php?" + qs.toString());

          await showResult(res);
        } catch (err) {
          document.querySelector("#result").textContent = "요청 실패";
          console.error(err);
        }
      });

    // POST: JSON 본문으로 입력값 전송
    document.querySelector("#postBtn")
      .addEventListener("click", async () => {
        const name = document.querySelector("#name").value;
        const age = Number(document.querySelector("#age").value);

        try {
          const res = await fetch("/user.php", {
            method: "POST",
            headers: {
              "Content-Type": "application/json"
            },
            body: JSON.stringify({ name, age })
          });

          await showResult(res);
        } catch (err) {
          document.querySelector("#result").textContent = "요청 실패";
          console.error(err);
        }
      });
  </script>
</body>
</html>
```

```php
<?php
header("Content-Type: application/json; charset=UTF-8");

if ($_SERVER["REQUEST_METHOD"] === "GET") {
    // GET: URL 쿼리에서 읽기
    $name = $_GET["name"] ?? "";
    $age = (int)($_GET["age"] ?? 0);

} elseif ($_SERVER["REQUEST_METHOD"] === "POST") {
    // POST: JSON 요청 본문에서 읽기
    $data = json_decode(
        file_get_contents("php://input"),
        true
    );

    $name = $data["name"] ?? "";
    $age = (int)($data["age"] ?? 0);

} else {
    http_response_code(405);
    header("Allow: GET, POST");

    echo json_encode(["message" => "지원하지 않는 요청 방식"]);
    exit;
}

// 공통 응답
echo json_encode(
    ["message" => $name . " 수신 완료"],
    JSON_UNESCAPED_UNICODE
);
```